PNAS RESEARCH ARTICLE | CLASSIFICATION

Fraud Detection using Network Features for Synthetic Data: A Comparative Study

Josie Goodson, Renxin (Jessica) Li, Moses Fanadzhyan, Preeti Karthikeyan, Reina Schmoock

Credit card fraud detection requires automated systems capable of identifying suspicious transactions. This study compares supervised and unsupervised learning approaches using intrinsic and network-based features derived from a large synthetic dataset. We evaluate Random Forest, Isolation Forest, K-means clustering, XGBoost, and neural network models using standard performance metrics. Results show that the Random Forest model achieves the strongest performance, with other approaches also demonstrating promising results.

Fraud detection | APATE | BiRank | Bipartite | Random Forest

Significance

Millions of people in the US fall victim to fraud annually. Improving fraud detection systems disincentivizes fraudsters. It also means that fraud victims can be notified of fraud faster, speeding up their recovery process. Many have developed fraud detection networks, but focusing on which features are the most significant decreases the time and money needed to detect fraud. Financial institutions have large operational costs due to investigating false-positive alerts.

Credit card fraud remains a persistent challenge for financial institutions and consumers. As criminals develop more sophisticated methods to evade detection systems, the need for automated and accurate fraud detection has become increasingly important.

This study evaluates supervised and unsupervised learning approaches for credit card fraud detection using intrinsic behavioral features and network-based features derived from the Anomaly Prevention using Advanced Transaction Exploration (APATE) framework (3), which models cardholder–merchant relationships. We introduce a BiRank-augmented initialization that incorporates structural information from the transaction network.

Random Forest, XGBoost, K-means clustering, Isolation Forest, and neural networks are evaluated on a large synthetic transaction dataset. The supervised tree-based models achieve the strongest performance. Isolation Forest shows promise as an unsupervised alternative, detecting substantial fraud without labeled data, whereas k-means clustering performs poorly due to difference in cluster structures.

The results highlight the value of adding network derived methods to intrinsic features. Section 2 describes the dataset and problem, Section 3 presents the feature engineering methodology, Section 4 introduces the predictive models, Section 5 reports experimental results, and Section 6 concludes with discussion and findings.

Related Literature

Credit card fraud is commonly detected using machine learning models while some studies instead focus on graph-based methods to analyze relationships within transaction networks. For example, (1) detects credit card fraud using neural networks trained on transactional and behavioral attributes. Similarly, (3) uses recency, frequency, and monetary value features within Random Forest models to fraudulent transactions. In contrast, other studies such as (4) and (2) use graph-based approaches by constructing bipartite networks and assigning risk scores to between connected nodes to identify suspicious interactions. Our work builds on these ideas by incorporating behavioral and transactional attributes with network based features to more accurately analyze individual cardholder behavior within the overall transaction network and improve model performance.

Data and Problem Setup

We use a synthetic credit card transaction dataset generated with Sparkov. Let each transaction in the synthetic data be $\tau_i = (c_i, m_i, a_i, t_i, \mathbf{v}_i)$, where c_i is the cardholder, m_i is the merchant, a_i is the transaction amount, t_i is the timestamp, and $\mathbf{v}_i$ collects the remaining transaction attributes (see Table 3).

The full transaction set is $\mathcal{T} = \{\tau_i\}_{i=1}^{N}$, where N is the total number of transactions ($13,843,20$ transactions in the expanse of 1.6 years). For each transaction i, let $\underline{t}_i$ denote midnight at the start of the calendar day containing t_i, and define the available historical information by $\mathcal{H}_i = \{\tau_j \in \mathcal{T} : t_j < \underline{t}_i\}$. We then extract a transaction-level feature vector $x_i = \phi(\tau_i, \mathcal{H}_i) \in \mathbb{R}^p$, where ϕ denotes the mapping that produces p features from transaction i and its prior history. Specifically, we decompose $x_i = (x_i^{\text{intrinsic}}, x_i^{\text{network}})$.

We partition the transaction indices into disjoint subsets I_{train} and I_{test}, where I_{test} corresponds to the one-month holdout period used for evaluation. Thus, we seek to understand the classification of fraudulent transactions using both supervised and unsupervised models. Unsupervised models are fitted on the full feature set $\mathcal{X} = \{x_i\}_{i=1}^{N}$. For supervised learning, labels are only available for indices in I_{train}, and we define the labeled training set as $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i \in I_{\text{train}}}$, where $y_i \in \{0, 1\}$ indicates whether transaction i is fraudulent. We then evaluate the trained classifier on the holdout transactions indexed by I_{test}, using their extracted feature vectors $\{x_i\}_{i \in I_{\text{test}}}$.

Feature Engineering

Intrinsic Features Extraction. The intrinsic feature vector $x_i^{\text{intrinsic}}$ consists of transaction-level features derived directly from τ_i and its prior history H_i, without using the network-based constructions introduced below. These include: (i) raw transaction attributes such as amount, merchant category, timestamp, and geographic coordinates; (ii) transformed amount features including $\log(1 + a_i)$ and ratios relative to historical cardholder or category medians; (iii) temporal indicators such as hour of day, day of week, weekend flags, and transaction recency; (iv) historical behavioral summaries including recent card and merchant transaction counts; and (v) geographic and mobility features. Geographic features include the geodesic distance between cardholder and merchant locations, its logarithmic transformation, the cardholder's historical average transaction distance, and deviations from this historical baseline. We also include state-level fraud rates and fraud rates defined over logarithmic distance bins to capture broader regional patterns. Together, these variables capture whether the current transaction is locally unusual in magnitude, timing, location, or recent behavioral context. A complete list of intrinsic features is provided in the supplementary information.

Network Feature Extraction. To capture fraud exposure at different temporal scales, we repeat the network construction over three lookback windows $r \in \{\text{ST}, \text{MT}, \text{LT}\}$ corresponding to short-, medium-, and long-term history, where $L_{\text{ST}} = 1$ day, $L_{\text{MT}} = 7$ days, and $L_{\text{LT}} = 30$ days. For a fixed window r, let

$$H_i = \{\tau_j \in \mathcal{T} : \underline{t}_i - L_r \leq t_j < \underline{t}_i\}$$

denote the set of historical transactions available for transaction i. Because H_i depends only on the calendar day containing t_i, all transactions occurring on the same day share the same historical window. For notational simplicity, we suppress the superscript $^{(r)}$ once the window is fixed. From H_i, we construct a weighted tripartite graph $G_i = (C_i \cup M_i \cup T_i, E_i)$, where C_i, M_i, and T_i denote the sets of cardholder, merchant, and transaction nodes appearing in H_i (3). Each historical transaction $\tau_j \in T_i$ connects to its cardholder c_j and merchant m_j, producing edges (c_j, τ_j) and (m_j, τ_j).To incorporate temporal recency, we assign exponentially decaying edge weights $w_j = e^{-\gamma_r h_j}$, where h_j is the elapsed time between transaction j and $\underline{t}_i$. Following (3), the decay parameters are chosen so that weights approach zero after one day, one week, and one month for the short-, medium-, and long-term windows, respectively. Accordingly, we set $\gamma_{\text{ST}} = 0.03$, $\gamma_{\text{MT}} = 0.004$, and $\gamma_{\text{LT}} = 0.0001$.

Let Q_i denote the weighted adjacency matrix of this tripartite graph, and let $Q_{i,\text{norm}}$ be its column-normalized version. Fraud influence is initialized through a starting vector $\mathbf{z_i}$, which assigns positive weights to confirmed fraudulent historical transactions by the same exponential decay scheme and zero to all other nodes. Exposure scores are then obtained by the iterative propagation

$$\xi_i^{(k)} = \alpha Q_{i,\text{norm}} \xi_i^{(k-1)} + (1 - \alpha)\mathbf{z_i} \qquad [1]$$

Let ξ_i denote the converged exposure vector obtained from the propagation in Eq. (1). The exposure score of a node v is given by component $\xi_i(v)$.

For each transaction i in the current day with cardholder c_i and merchant m_i, we define the cardholder exposure score and merchant exposure score as $\text{CCHScore}_i = \xi_i(c_i)$, $\text{MCScore}_i = \xi_i(m_i)$. Let $\mathcal{T}_{c_i,m_i} = \{\tau_j \in H_i : c_j = c_i, m_j = m_i\}$ denote the set of historical transactions between cardholder c_i and merchant m_i. If this set is nonempty, we let τ_i^* denote the most recent such historical transaction in the lookback window. TXScore_i for a current-day transaction i is either inherited from the most recent historical transaction with the same cardholder-merchant pair, or estimated from the pair's endpoints exposures when no such historical transaction exists. Thus, it is defined as

$$\text{TXScore}_i = \begin{cases} \xi_i(\tau_i^*), & \mathcal{T}_{c_i,m_i} \neq \varnothing, \\ \dfrac{\text{CCHScore}_i}{\sum_{\tau_j \in H_i:\, c_j = c_i} w_j + 1} & \\ + \dfrac{\text{MCScore}_i}{\sum_{\tau_j \in H_i:\, m_j = m_i} w_j + 1}, & \mathcal{T}_{c_i,m_i} = \varnothing. \end{cases} \qquad [2]$$

Together, Eq. (1) and Eq. (2) define the APATE-based network scores.

BiRank-Augmented Initialization. We introduce an APATE-BiRank variant (2). In the original APATE model, the restart vector $\mathbf{z}_i$ places nonzero mass only on confirmed fraudulent historical transactions, resulting in a sparse and purely label-driven initialization. This does not take into account the broader interaction structure among cardholders and merchants in H_i, even though such structure may indicate elevated exposure for transactions connected to suspicious parts of the network. Following prior transaction-network work that uses aggregate transferred amount as an edge weight (6), we treat cumulative transaction amount as a proxy for the strength of a historical cardholder–merchant interaction. This motivates using transaction amount to weight the cardholder-merchant edges in the BiRank graph, so that stronger historical interactions contribute more to the structural prior.

To supplement the APATE initialization with structural information from historical cardholder-merchant interactions, we compute BiRank scores on a weighted bipartite graph derived from the same history window H_i. For a fixed window r, let $B_i = (C_i \cup M_i, E_i^B)$ denote the weighted bipartite graph whose node sets are the cardholders C_i and merchants M_i appearing in H_i. For each pair $(c, m) \in C_i \times M_i$, we assign an edge weight

$$w_i(c, m) = \sum_{\tau_j \in H_i:\, c_j = c,\, m_j = m} \log(1 + a_j),$$

that is, the sum of $\log(1 + a_j)$ over all historical transactions in H_i within the given time window between cardholder c and merchant m. This defines a weighted biadjacency matrix W_i, whose (c, m)-entry is $w_i(c, m)$.

Let $d_i^C(c) = \sum_{m \in M_i} w_i(c, m)$, $d_i^M(m) = \sum_{c \in C_i} w_i(c, m)$ denote the weighted degrees of cardholder

and merchant nodes, and let D_i^C and D_i^M be the corresponding diagonal degree matrices. We then form the symmetrically normalized matrix

$$S_i = (D_i^C)^{-1/2} W_i (D_i^M)^{-1/2} \quad (2)$$

We initialize BiRank with uniform prior vectors over the cardholder and merchant nodes, since no node-specific prior information is imposed, letting $u_i^0 = \frac{1}{|C_i|}\mathbf{1}, p_i^0 = \frac{1}{|M_i|}\mathbf{1}$. The cardholder and merchant scores are then updated iteratively by

$$\begin{aligned} p_i^{(k)} &= \alpha_B S_i^\top u_i^{(k-1)} + (1-\alpha_B) p_i^0, \\ u_i^{(k)} &= \beta_B S_i p_i^{(k)} + (1-\beta_B) u_i^0. \end{aligned} \quad [3]$$

where $\alpha_B = \beta_B = 0.85$. After each iteration, both vectors are renormalized to sum to one. The iteration is terminated after at most 10 steps, or earlier if the ℓ_1 difference between successive iterates falls below 10^{-8}.

Let u_i and p_i denote the converged BiRank score vectors on C_i and M_i from the iterative updated in Eq. (3), respectively. For the current transaction i, we then extract the BiRank-based cardholder and merchant prior scores $\text{BiRankCCH}_i = u_i(c_i)$, $\text{BiRankMC}_i = p_i(m_i)$. These scores summarize the structural position of the current transaction's cardholder and merchant within the weighted cardholder–merchant network induced by H_i.

Since $\mathbf{z}_i^{\text{BR}}$ is defined on the node set of the tripartite graph G_i, the argument v denotes any node in $C_i \cup M_i \cup T_i$; in particular, v may be a cardholder node, a merchant node, or a transaction node. We then define a BiRank-augmented restart vector $\mathbf{z}_i^{\text{BR}} \in \mathbb{R}^{|C_i|+|M_i|+|T_i|}$ by

$$z_i^{\text{BR}}(v) = \begin{cases} z_i(v), & v \in T_i, \\ \text{BiRankCCH}_i, & v = c_i, \\ \text{BiRankMC}_i, & v = m_i. \end{cases} \quad [4]$$

That is, we retain the original APATE initialization on historical transaction nodes and replace the zero entries at the current cardholder and merchant nodes by the corresponding BiRank prior scores. We then apply the same propagation procedure as in Eq. (1) but with the BiRank-augmented restart vector from Eq. (4) in place $\mathbf{z}_i$. Let $\xi_{i,\text{BR}}$ denote the converged exposure vector. The APATE-BiRank features $\text{CCHScore}_i^{\text{BR}}$, $\text{MCScore}_i^{\text{BR}}$, and $\text{TXScore}_i^{\text{BR}}$ are then defined analogously to Eqs. Eq. (1)–Eq. (2), but using $\xi_{i,\text{BR}}$ instead of ξ_i.

In addition, BiRankCCH_i and BiRankMC_i are retained as separate network features. Collectively, Collectively, the APATE scores from Eq. (1)–Eq. (2), the BiRank prior scores from Eq. (3), and the APATE-BiRank scores induced by Eq. (4) constitute x_i^{network}.

Predictive Models

Random Forest. We trained a Random Forest classifier on the transaction-level feature vectors x_i. Numeric variables were median-imputed to handle missing values in skewed features, while categorical variables were imputed with their most frequent value and one-hot encoded. To address the strong class imbalance in fraud detection, class-weighted training was used so that errors on fraud cases received greater emphasis. The final model used 100 trees with minimum split size 20 and minimum leaf size 5. The classification threshold was tuned on a time-based validation set to maximize recall subject to a false-positive-rate constraint of at most 1%, and then evaluated on the one-month holdout test set.

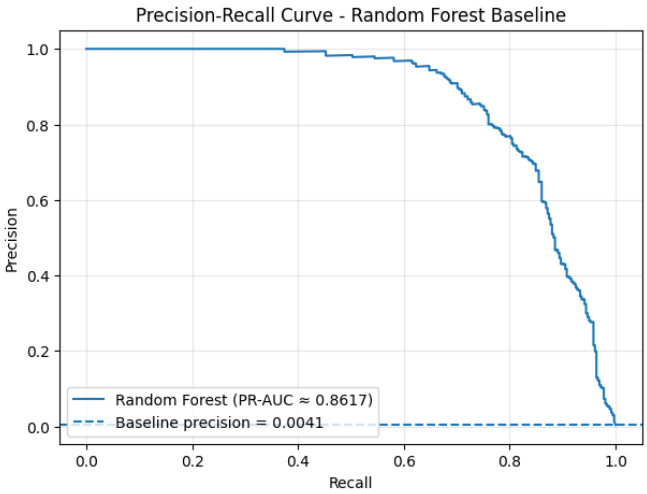


Fig. 1. Precision–recall curve for the Random Forest model on the holdout test set, with PR-AUC $= 0.8617$. The dashed line indicates the baseline precision given by the fraud prevalence.

XGBoost. The feature vectors x_i were also used to train a supervised XGBoost classifier. XGBoost builds an ensemble of decision trees sequentially, where each new tree corrects the errors of the previous ones. Model complexity was controlled through several hyperparameters: max_depth $= 6$ limits tree depth, learning_rate $= 0.05$ controls the contribution of each new tree, and subsample $= 0.8$ and colsample_bytree $= 0.8$ introduce randomness to reduce overfitting. The parameter min_child_weight $= 5$ prevents splits based on very small subsets of data, while gamma $= 0.5$ requires a minimum improvement in the objective function before a split is created.

Isolation Forest. Isolation Forest was used as an unsupervised anomaly detection method that identifies unusual observations through random partitioning of the feature space. Given transaction feature vectors $X = \{x_1, x_2, ..., x_n\}$, the algorithm constructs an ensemble of random trees and measures the path length required to isolate each observation. Transactions requiring fewer splits are considered more anomalous. The resulting anomaly score is used to rank transactions, and those with the lowest scores are predicted as fraudulent.

Neural Networks. A feedforward Multi-Layer Perceptron was trained to evaluate transactions using two engineered dimensions: temporal context (hour of day) and proportional magnitude (amount-to-card median ratio). To prevent target leakage, the category-specific Z-score magnitude used to define anomalies was excluded from the input space. The input features X_i were standard-scaled with mean imputation for missing values. The network contained a single hidden layer with 10 neurons and was trained using an anomaly target defined by Z-score magnitude exceeding 2.0. The classification threshold on the predicted probabilities was adjusted to emphasize recall while constraining review volume to the top 1% of the highest-risk transactions.

K-Means Clustering. K-means clustering was used as an unsupervised method that partitions data by minimizing intra-cluster variance. For fraud detection, we set $k = 2$ to separate transactions into two clusters, with the smaller cluster predicted as fraudulent. As shown in Eq. (5), given transactions $X = \{x_1, x_2, ..., x_n\}$, k-means finds centroids μ_1 and μ_2 and assigns each transaction to the nearest centroid. Only quantitative features were used in clustering; categorical variables and the `is_fraud` label were excluded from training. The `is_fraud` column

Table 1. Overall comparison of predictive models on the one-month holdout test set.

Model	ROC-AUC	PR-AUC	Precision	Recall	F_1
Random Forest	.9953	.8618	.8168	.7598	.7873
XGBoost	.8727	.6434	.8613	.7546	.7994
K-means	.6483	.0055	.0044	.2179	.0086
Isolation Forests	.8706	.641	.05	.47	.1
Neural Network	.6302	.0004	.0011	.0909	.0023

All metrics are reported on the one-month holdout test set. Precision, recall, and F1 in Eq. (9) refer to the fraud class.

Table 2. Random Forest performance using different feature subsets on the one-month holdout test set.

Feature set	ROC-AUC	PR-AUC	Fraud precision in Eq. (9)
Network only	0.7784	0.2448	0.5676
Intrinsic only	0.9949	0.8241	0.7113
Full feature set	0.9953	0.8618	0.8267

Precision refers to the fraud class at threshold 0.5.

was retained only for post-hoc evaluation of cluster assignments. A transaction identifier column was included solely for row tracking and excluded from the clustering procedure.

Results

The fraud detection task is highly imbalanced, with fraudulent transactions accounting for only 0.58% of the training set and 0.41% of the test set. Because such strong class imbalance makes overall accuracy less informative, we focus on fraud-class precision, recall and F_1 scores. We also report ROC-AUC and PR-AUC to evaluate ranking quality, with PR-AUC being more informative in the imbalanced setting.

Overall Model Comparison. Table 1 summarizes the predictive performance of all models on the one-month holdout test set. The strongest results are obtained by the supervised tree-based models, Random Forest and XGBoost, which achieve the highest ROC-AUC and PR-AUC values and substantially outperform the unsupervised methods in fraud-class precision, recall, and F_1. Among them, Random Forest provides the best overall performance, achieving the highest PR-AUC (Figure 1) and the strongest fraud-class precision.

The unsupervised methods perform considerably worse. K-means clustering shows weak performance across all metrics, indicating that simple partitioning of the feature space is insufficient to isolate the rare fraud class. Isolation Forest performs better due to its anomaly-detection design, but its fraud precision remains low because many anomalous transactions are legitimate. The neural network configuration also underperforms the tree-based models in this setting.

Overall, these results suggest that fraud detection in this dataset benefits from supervised models that leverage labeled data and capture nonlinear relationships among the engineered features.

Effect of Network Features. To assess the contribution of feature groups, we compared Random Forest models trained on intrinsic features only, network features only, and the full feature set (Table 2). The intrinsic-only model already achieves strong performance with PR-AUC 0.8241, indicating that transaction descriptors such as amount, time, location, and recent behavioral summaries capture much of the predictive signal. The network-only model

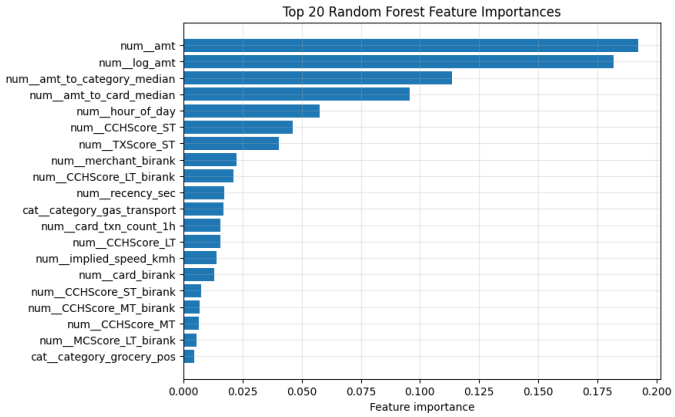


Fig. 2. Top 20 Random Forest feature importances for the full feature set.

performs substantially worse, although its ROC-AUC and PR-AUC remain above random.

Feature-importance analysis in Figure 2 shows that amount-based and temporal variables remain among the strongest predictors. However, once the APATE and BiRank variables are included, several network-derived features—particularly short-term cardholder and transaction exposure scores and the BiRank-based merchant and card scores—also rank among the top predictors. This suggests that network features provide complementary structural information that improves fraud ranking when combined with intrinsic transaction descriptors.

Conclusion and Discussion

The experimental results demonstrate clear differences between supervised and unsupervised approaches to fraud detection. Among the models evaluated, Random Forest achieved the strongest overall performance, with a ROC-AUC of 0.9953 and balanced precision and recall, indicating that it was highly effective at separating fraudulent from legitimate transactions. The XGBoost model also performed well, achieving strong predictive accuracy and confirming that boosted tree ensembles are well suited for capturing nonlinear relationships in transaction data.

More specifically, the feature-importance analysis shows that several APATE and BiRank variables rank among the strongest predictors once they are added to the model. This suggests that BiRank captures meaningful structural information about historical cardholder–merchant relationships and helps refine the fraud ranking produced by the intrinsic features. Thus, the main contribution of BiRank in our setting is as an augmentation to the network feature representation, not as a replacement for local transaction descriptors.

Because fraudulent transactions are rare, evaluation must focus on PR-AUC and fraud-class precision, recall, and F_1, rather than on accuracy alone (such as ROC-AUC). Future work could explore richer BiRank priors, broader hyperparameter tuning, and alternative threshold-selection strategies that better reflect practical costs of false positives and missed frauds.

Limitations

Several limitations should be noted. Our experiments were conducted on synthetic transaction data, which may not fully reflect the complexity and noise of real-world fraud patterns. In addition, time constraints limited the number of models explored, and computational constraints restricted the scale of the experiments. As a result, the performance and scalability of the proposed network-based features should be further evaluated on larger, real-world datasets.

Acknowledgments

We are grateful towards Dr. Paolo Climaco in consultation of appropriate machine learning methods and network related features.

References

1. Alatawi, M.N (2025) *"Detection of fraud in IoT based credit card collected dataset using machine learning* ScienceDirect. https://www.sciencedirect.com/science/article/pii/S2666827024000793
2. He X, Gao M, Kan M, & Wang D. (2017) *"BiRank: Towards Ranking on Bipartite Graphs* IEEE https://arxiv.org/pdf/1708.04396
3. Vlasselaer, V.V, Bravo, C., Caelen, O., Eliassi-Rad, T., Akoglu, L., Snoeck, M., & Baesens, B. (2015) *"A novel approach for automated credit card transaction fraud detection using network-based extensions.* ScienceDirect. https://www.sciencedirect.com/science/article/pii/S0167923615000846
4. Wu Y., Xu W,, & Li J. (2021) *"Fraudulent traffic detection in online advertising with bipartite graph propagation algorithm* ScienceDirect. https://www.sciencedirect.com/science/article/pii/S0957417421009775
5. Kartik Shenoy (2021) *"Credit Card Fraud Detection Dataset" (Synthetic)* Kaggle. https://www.kaggle.com/datasets/kartik2112/fraud-detection
6. Saxena, A., Pei, Y., Veldsink, J., van Ipenburg, W., Fletcher, G., & Pechenizkiy, M. (2021) *The Banking Transactions Dataset and its Comparative Analysis with Scale-free Networks* arXiv. https://arxiv.org/abs/2109.10703

Author Contributions

Moses Fanadzhyan proposed the research question how to better create an algorithm which detects fraud. Preeti Karthikeyan acquired a synthetic data set for which Renxin(Jessica) Li adapted all twenty network related features from APATE and BiRank algorithms. Moses Fanadzhyan, Reina Schmoock and Josie Goodson extracted geographical and median spend features from the synthetic data, where then Moses Fanadzhyan and Renxin(Jessica) Li extracted intrinsic features except for geographic ones and merged all intrinsic and network related features for use by the algorithms. Josie Goodson created a neural network algorithm; Reina Schmoock created a XGBoost algorithm; Preeti Karthikeyan created an isolation forest algorithm; Renxin(Jessica) Li created a Random Forest algorithm; and Moses Fanadzhyan created a K-Means Clustering algorithm. All five ran the new merged data through their respective algorithms and reported/compared their findings. All five then wrote their respective machine learning models section in the paper, as well as their findings. Moses Fanadzhyan wrote the Introduction, Abstract and Acknowledgements. Reina Schmoock wrote the Related Literature. Renxin(Jessica) Li wrote the Data and Problem Setup, Feature Engineering, and Results, as well as figures and tables used in paper. Preeti Karthikeyan worked on the Conclusion and Discussion. Moses Fanadzhyan fixed formatting issues.

Scientific Replication

The following Github repository will allow for replicating our findings. Read the respective Readme files to appropriately run the files: https://github.com/mosesf1/M168.

Supplementary Materials

0.1. K-Means Formulation. Let $x_i \in \mathbb{R}^p$ denote the feature vector for transaction t_i, where p represents only the quantitative features used for clustering. K-means with $k = 2$ partitions the data into two clusters C_1 and C_2 by minimizing the within-cluster sum of squares:

$$\min_{\mu_1,\mu_2} \sum_{j=1}^{2} \sum_{x_i \in C_j} ||x_i - \mu_j||^2 \quad [5]$$

where μ_j represents the centroid of cluster C_j. The algorithm iterates between two steps until convergence:

1. **Assignment**: Each transaction is assigned to the nearest centroid:

$$C_j = \{x_i : ||x_i - \mu_j||^2 \leq ||x_i - \mu_k||^2, k \neq j\} \quad [6]$$

2. **Update**: Centroids are recomputed as the mean of assigned points:

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i \quad [7]$$

After convergence, the cluster with fewer transactions is designated as the fraud cluster C_{fraud}. Each transaction is classified based on its cluster assignment:

$$\hat{y}_i = \begin{cases} 1 & \text{if } x_i \in C_{fraud} \\ 0 & \text{otherwise} \end{cases} \quad [8]$$

0.2. K-Means Results. The k-means model was evaluated on the one-month holdout test set, with the `is_fraud` column used only for performance assessment after clustering. The clustering approach achieved an accuracy of 79.42%, with precision of 0.44%, recall of 21.79%, and an F1-score of 0.86% for fraud detection.

The ROC-AUC of 52.93% indicates performance barely above random guessing (50%), while the PR-AUC of 1.13% reflects the challenge of fraud detection given the class imbalance. Although recall showed the model identifying approximately one-fifth of actual fraudulent transactions, the precision of 0.44% indicates that less than 0.5% of transactions flagged as fraud were actually fraudulent, resulting in an extremely high false positive rate. The F1-score of 0.86% reflects the poor overall performance despite the modest recall.

These results suggest that with $k = 2$, the clustering algorithm failed to separate fraudulent from legitimate transactions effectively. The near-random ROC-AUC confirms that the distance-based scoring does not reliably distinguish between classes, while the extremely low precision indicates that even when the algorithm flags transactions as fraudulent, it is incorrect more than 99.5% of the time. Consequently, no meaningful conclusions can be drawn about fraud detection from this unsupervised approach, as the cluster structure learned from the data does not correspond to the presence or absence of fraudulent activity.

XGBoost Model. XGBoost improves prediction performance by combining the outputs of many decision trees. Each tree separates the data into smaller regions based on feature values, allowing the model to capture nonlinear relationships among variables such as transaction amount, geographic distance, and temporal behavior. The boosting process builds trees sequentially, with each new tree focusing on transactions that were previously misclassified. By gradually correcting prediction errors across many iterations, the model learns complex patterns associated with fraudulent activity.

XGBoost Formulation. Let $x_i \in \mathbb{R}^p$ denote the feature vector for transaction t_i, and $y_i \in \{0, 1\}$ indicate whether the transaction is fraudulent. XGBoost produces a prediction by adding together the outputs of several decision trees,

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i),$$

where each function f_k represents a decision tree. The model is trained by minimizing a regularized objective function,

$$\mathcal{L} = \sum_{i=1}^{n} \ell(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k),$$

where $\ell(\cdot)$ measures prediction error and $\Omega(\cdot)$ reduces complexity to prevent overfitting.

0.3. Isolation Forest for Fraud Detection. To detect anomalous credit card transactions, we use the Isolation Forest algorithm, an unsupervised anomaly detection method designed to identify observations that deviate significantly from normal behavioral patterns. Unlike supervised classifiers, Isolation Forest does not directly learn a mapping between features and labels. Instead, it isolates anomalies by recursively partitioning the feature space using randomly generated decision trees.

Given a dataset of transaction features

$$X = \{x_1, x_2, \ldots, x_n\}, \quad x_i \in \mathbb{R}^d,$$

Isolation Forests construct an ensemble of random trees by repeatedly selecting a random feature and a random split value. Anomalies tend to be isolated with fewer splits because they lie in sparse regions of the feature space.

For each observation x, the algorithm computes the path length $h(x)$, which represents the number of splits required to isolate that point in a tree. Averaging this value across all trees yields the expected path length $E[h(x)]$. The anomaly score is then defined as

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}},$$

where $c(n)$ is the average path length of unsuccessful searches in a binary search tree:

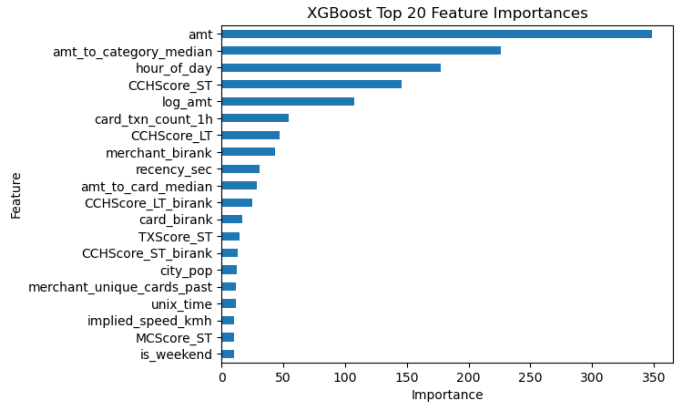
$$c(n) = 2H(n-1) - \frac{2(n-1)}{n},$$

and $H(n)$ denotes the harmonic number. Observations with shorter expected path lengths receive larger anomaly scores and are therefore more likely to be considered anomalous.

The algorithm produces a decision function score $f(x)$ where lower values correspond to more anomalous observations. To identify potential fraudulent transactions, we rank transactions according to their anomaly scores and classify the most extreme observations as fraud candidates. Specifically, if q_α denotes the α-quantile of the anomaly score distribution, we define the predicted fraud indicator as

$$\hat{y}_i = \begin{cases} 1 & \text{if } f(x_i) \leq q_\alpha \\ 0 & \text{otherwise} \end{cases}$$

where α determines the proportion of transactions flagged as anomalies.



0.4. Evaluation Metrics.

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP}, \\ \text{Recall} &= \frac{TP}{TP + FN}, \\ F_1 &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \end{aligned} \quad [9]$$

0.5. Intrinsic Features. See Table 3.

Table 3. Intrinsic transaction-level features grouped by type.

Category	Feature	Description
Original attributes (5)	trans_date_trans_time	Transaction timestamp in calendar date-time format.
	cc_num	Cardholder identifier (credit card number).
	merchant	Merchant identifier at which the transaction occurred.
	category	Merchant category of the transaction.
	amt	Raw transaction amount.
	first, last	Cardholder first and last name.
	gender, dob, job	Cardholder demographic attributes: gender, date of birth, and occupation.
	street, city, state, zip, lat, long, city_pop	Cardholder address, geographic coordinates, and city population.
	trans_num	Unique transaction identifier.
	unix_time	Transaction timestamp in Unix time.
	merch_lat, merch_long	Merchant geographic coordinates.
	is_fraud	Fraud label, equal to 1 for fraudulent transactions and 0 otherwise.
Amount features	amt	Raw transaction amount.
	log_amt	Log-transformed transaction amount, computed as $\log(1 + \texttt{amt})$.
	card_past_median_amt	Past-only median transaction amount for the same cardholder.
	amt_to_card_median	Ratio of the current amount to the cardholder's past median amount.
	amt_to_card_median_missing	Indicator for whether the past card-level median is unavailable.
	category_past_median_amt	Past-only median transaction amount for the same merchant category.
	amt_to_category_median	Ratio of the current amount to the category-level past median amount.
Short-term card behavior	card_txn_count_1h	Number of prior transactions by the same cardholder within the previous hour.
	card_unique_merchants_1h	Number of distinct merchants visited by the same cardholder within the previous hour.
	recency_sec	Time elapsed since the previous transaction made by the same cardholder.
Merchant history	merchant_txn_count_past	Number of previous transactions observed for the same merchant.
	merchant_unique_cards_past	Number of distinct cardholders previously observed at the same merchant.
Temporal context	hour_of_day	Hour of day extracted from the transaction timestamp.
	day_of_week	Day of week extracted from the transaction timestamp.
	is_weekend	Indicator equal to 1 if the transaction occurred on a weekend.
Geographic and mobility	dist_home_merchant_km	Distance between the cardholder's home location and the merchant location.
	AvgCardDist_km_	The average distance between the home location and merchant location for all transactions of a given cardholder
	DistRatio	Distance between the home location and merchant location of a given transaction to the typical (average) distance for that cardholder
	dist_prev_merchant_km	Distance between the current merchant and the previous merchant used by the same cardholder.
	implied_speed_kmh	Implied travel speed between consecutive merchant locations.
	LogDistance_km_	Distance between the home location and the merchant location scaled logarithmically
	BinFraudRate	Fraud rate of each bin where each transaction is assigned a bin for local, regional, long-distance, and international transactions
	fraud_rate	Fraudulent transaction in a state out of all transactions in the given state